

4.3 Guía de Funciones: Auditoría de Equidad (Fairness Audit)

R — Auditoría de sesgo por grupos

```
# AUDITORÍA DE FAIRNESS – ejemplo con dataset de crédito simulado
library(caret)
library(dplyr)
library(ggplot2)

set.seed(42)
n <- 1000
credito <- data.frame(
  edad = round(rnorm(n, 38, 10)),
  ingresos = rexp(n, 0.000003),
  deuda_ratio = pmin(1, pmax(0, rnorm(n, 0.35, 0.2))),
  sexo = factor(sample(c("M", "F"), n, replace=TRUE, prob=c(0.55, 0.45)))
)

# Introducir sesgo histórico: mujeres aprobadas menos históricamente
lp <- -1 + 0.000001*credito$ingresos - 1.5*credito$deuda_ratio +
  0.02*credito$edad - 0.4*(credito$sexo=="F") # sesgo histórico
credito$aprobado_historico <- factor(rbinom(n, 1, 1/(1+exp(-lp))), labels=c("No", "Si"))

# 1. ENTRENAR MODELO (aprenderá el sesgo)
idx <- createDataPartition(credito$aprobado_historico, p=0.8, list=FALSE)
train <- credito[idx,]; test <- credito[-idx,]

# Modelo SIN variable de sexo (¿elimina el sesgo?)
m_sin_sexo <- train(aprobado_historico ~ edad + ingresos + deuda_ratio,
  data=train, method="glm", family="binomial",
  preProcess=c("center", "scale"))

# Modelo CON variable de sexo
m_con_sexo <- train(aprobado_historico ~ ., data=train,
```

```

        method="glm", family="binomial",
        preProcess=c("center","scale"))

# 2. AUDITORÍA POR GRUPO
auditar_fairness <- function(modelo, test_data, grupo_var, positive="Si") {
  probs <- predict(modelo, test_data, type="prob")[, positive]
  pred <- predict(modelo, test_data)

  resultados <- test_data %>%
    mutate(prob = probs, pred = pred,
           real = aprobado_historico) %>%
    group_by(!!sym(grupo_var)) %>%
    summarise(
      n = n(),
      tasa_aprobacion = mean(pred == "Si"), # Paridad demográfica
      tasa_aprobacion_real = mean(real == "Si"),
      recall = sum(pred=="Si" & real=="Si") / sum(real=="Si"), # Igualdad de oportunidad
      precision = sum(pred=="Si" & real=="Si") / sum(pred=="Si"),
      accuracy = mean(pred == real),
      .groups = "drop"
    )
  resultados
}

cat("=== Modelo SIN sexo ===")
print(auditar_fairness(m_sin_sexo, test, "sexo"))

cat("=== Modelo CON sexo ===")
print(auditar_fairness(m_con_sexo, test, "sexo"))

# 3. DISPARATE IMPACT RATIO (DIR)
# DIR = tasa_aprobacion_grupo_minoritario / tasa_aprobacion_grupo_mayoritario
# DIR < 0.8 sugiere discriminación (regla del 80% de EEOC en EE.UU.)
calcular_dir <- function(audit_result, grupo_ref) {
  tasa_ref <- audit_result$tasa_aprobacion[audit_result[[names(audit_result)[1]]] == grupo_ref]
  audit_result %>%
    mutate(DIR = tasa_aprobacion / tasa_ref,
           Alerta = ifelse(DIR < 0.8, "POSIBLE DISCRIMINACIÓN", "OK"))
}

audit <- auditar_fairness(m_sin_sexo, test, "sexo")
print(calcular_dir(audit, "M"))

```

Python — fairlearn

```
# fairlearn es la librería de Microsoft para auditoría de fairness
# pip install fairlearn

import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Simular datos con sesgo
np.random.seed(42)
n = 1000
df = pd.DataFrame({
    'edad': np.random.normal(38, 10, n),
    'ingresos': np.random.exponential(300000, n),
    'deuda_ratio': np.clip(np.random.normal(0.35, 0.2, n), 0, 1),
    'sexo': np.random.choice(['M', 'F'], n, p=[0.55, 0.45])
})

lp = -1 + df['ingresos']*0.000001 - 1.5*df['deuda_ratio'] + 0.02*df['edad'] - 0.4*(df['sexo']=='F')
df['aprobado'] = (np.random.uniform(size=n) < 1/(1+np.exp(-lp))).astype(int)

X = df[['edad', 'ingresos', 'deuda_ratio']]
y = df['aprobado']
sexo = df['sexo']

X_train, X_test, y_train, y_test, s_train, s_test = train_test_split(
    X, y, sexo, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

modelo = LogisticRegression(max_iter=1000)
modelo.fit(X_train_s, y_train)

# Auditoría manual por grupo
y_pred = modelo.predict(X_test_s)
test_df = pd.DataFrame({'real': y_test.values, 'pred': y_pred, 'sexo': s_test.values})

audit = test_df.groupby('sexo').agg(
    n=('real', 'count'),
```

```
tasa_aprobacion=('pred','mean'),
tasa_real=('real','mean'),
recall=lambda x: ((x['pred']==1) & (x['real']==1)).sum() / (x['real']==1).sum() if (x['real']==1).sum() > 0 else 0
accuracy=lambda x: (x['pred'] == x['real']).mean()
).reset_index()
print(audit)

# Disparate Impact Ratio
tasa_m = audit.loc[audit['sexo']=='M', 'tasa_aprobacion'].values[0]
audit['DIR'] = audit['tasa_aprobacion'] / tasa_m
audit['Alerta'] = audit['DIR'].apply(lambda x: 'POSIBLE DISCRIMINACION' if x < 0.8 else 'OK')
print(audit[['sexo', 'tasa_aprobacion', 'DIR', 'Alerta']])
```